

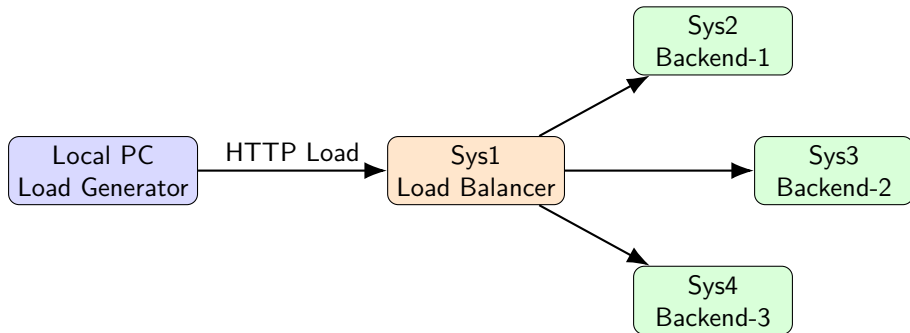
Building a Load Balancer in Go

Reverse Proxy, Round-Robin Scheduling, Health Checks and Performance Experiments

August 21, 2026

What Will We Build?

By the end of this implementation, we will have a complete distributed experiment:



The system demonstrates:

- A reverse proxy written in Go
- Round-robin request distribution
- Backend health checking
- Synthetic delays and failures

Go Installation on Ubuntu

Install Go before building and running the project

- 1 Update packages:

```
sudo apt update
```

- 2 Install Go using Ubuntu's package:

```
sudo apt install golang-go
```

- 3 Verify the installation:

```
go version
```

- 4 Check Go environment:

```
go env GOPATH  
go env GOROOT
```

Alternative: Install a specific/latest Go release from <https://go.dev/dl/>

Go Installation Resources

Recommended installation guides

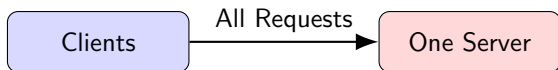
- **Official Go installation guide:**
<https://go.dev/doc/install>
- **Official Go downloads:**
<https://go.dev/dl/>
- **Official Go Wiki – Ubuntu:**
<https://go.dev/wiki/Ubuntu>
- **Ubuntu for Developers – Go setup:**
<https://documentation.ubuntu.com/ubuntu-for-developers/howto/go-setup/>

For this project:

```
go version
go build ./...
```

Why Do We Need a Load Balancer?

Suppose one server receives all incoming requests.

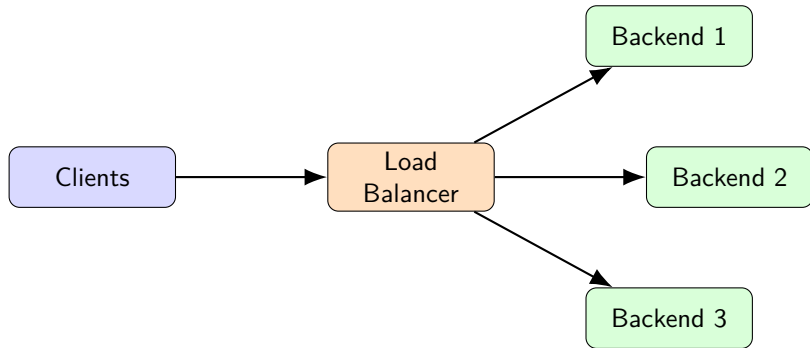


Problems:

- Server can become overloaded.
- A single server failure stops the service.
- CPU and memory are not shared across machines.
- Scaling requires manual changes.

Idea: place a Load Balancer in front of multiple backend servers.

Basic Load Balancing Architecture



The Load Balancer:

- 1 Receives a client request.
- 2 Selects a healthy backend.
- 3 Forwards the request.
- 4 Receives the backend response.
- 5 Returns the response to the client.

Step 1: Create a Simple Backend Server

Every backend server uses Go's standard HTTP library.

Basic idea:

```
mux := http.NewServeMux()

mux.HandleFunc("/health", healthHandler)
mux.HandleFunc("/", requestHandler)

http.ListenAndServe(":8081", mux)
```

Each backend can be started with:

```
go run ./backend -name backend-1 -port 8081
```

The same program is deployed to multiple machines with different names.

Backend Endpoint: /health

The Load Balancer needs a simple method to check whether a backend is alive.

```
mux.HandleFunc("/health",
    func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(http.StatusOK)
        fmt.Fprintln(w, "ok")
    })
```

The Load Balancer periodically sends:

```
GET http://BACKEND_IP:8081/health
```

If the response succeeds:

Backend Alive = true

Otherwise:

Backend Alive = false

Backend Endpoint: Normal Request Processing

The root endpoint processes actual workload requests:

```
mux.HandleFunc("/", func(  
    w http.ResponseWriter,  
    r *http.Request) {  
  
    id := requestID.Add(1)  
    total.Add(1)  
  
    // Optional delay  
    // Optional failure  
  
    json.NewEncoder(w).Encode(response{  
        Backend: *name,  
        RequestID: id,  
        Message: "ok",  
    })  
})
```

The response tells us which backend processed the request.

Example:

```
{  
  "backend": "backend-2",  
  "request_id": 125,  
  "delay_ms": 0,  
  "message": "ok"  
}
```

Simulating Backend Delay

Real servers are not always equally fast.
We can simulate processing delay using:

```
delay, err := time.ParseDuration(rawDelay)

if delay > 0 {
    time.Sleep(delay)
}
```

A client request can contain:

```
GET /?delay=100ms
GET /?delay=500ms
GET /?delay=1s
```

This helps us study:

- Latency increase
- Load Balancer timeout
- Request dropout
- Backend overload behaviour

Simulating Backend Failures

The backend can also generate synthetic failures.

Manual failure:

```
GET /?fail=true
```

Random failure:

```
if *failureRate > 0 &&  
    rand.Float64() < *failureRate {  
  
    failures.Add(1)  
  
    http.Error(  
        w,  
        "random synthetic failure",  
        http.StatusServiceUnavailable)  
  
    return  
}
```

This allows controlled failure experiments without physically destroying a server.

Step 2: Define the Backend Structure

The Load Balancer stores information about each backend.

```
type Backend struct {  
    URL      *url.URL  
    Alive     atomic.Bool  
    InFlight  atomic.Int64  
}
```

- URL: address of the backend
- Alive: current health status
- InFlight: number of active requests

Atomic variables are important because many HTTP requests may be processed concurrently.

Why Use Atomic Operations?

The Load Balancer is concurrent.

Multiple goroutines can simultaneously:

- Select backends
- Update counters
- Change health state
- Count requests

Unsafe approach:

```
counter = counter + 1
```

Thread-safe approach:

```
counter.Add(1)
```

The code uses:

- `atomic.Uint64`
- `atomic.Int64`
- `atomic.Bool`

This avoids data races in shared counters and backend state.

Step 3: Parse the Backend List

The Load Balancer receives backend URLs as a command-line argument.

Example:

```
-backends http://SYS2:8081,  
          http://SYS3:8081,  
          http://SYS4:8081
```

The code parses the comma-separated list:

```
parts := strings.Split(raw, ",")  
  
for _, part := range parts {  
    u, err := url.Parse(  
        strings.TrimSpace(part))  
  
    b := &Backend{URL: u}  
    b.Alive.Store(true)  
  
    backends = append(backends, b)  
}
```

Now the Load Balancer has a list of all backend servers.

Step 4: Implement Round-Robin Selection

The Load Balancer maintains an atomic counter:

```
next atomic.Uint64
```

Backend selection:

```
func (lb *LoadBalancer)
nextBackend() *Backend {

    n := len(lb.backends)

    for i := 0; i < n; i++ {
        index := lb.next.Add(1) % uint64(n)
        b := lb.backends[index]

        if b.Alive.Load() {
            return b
        }
    }

    return nil
}
```

The counter continuously moves through the backend array.

How Round-Robin Works

Suppose we have:

B_1, B_2, B_3

Requests are assigned approximately as:

Request	1	2	3	4	5	6	7	8	9
Backend	B_1	B_2	B_3	B_1	B_2	B_3	B_1	B_2	B_3

If Backend-2 is unhealthy:

$B_1 \rightarrow B_3 \rightarrow B_1 \rightarrow B_3$

Therefore, the algorithm combines:

Round-Robin + Health Awareness

Step 5: Add Health Checking

The Load Balancer runs a background goroutine:

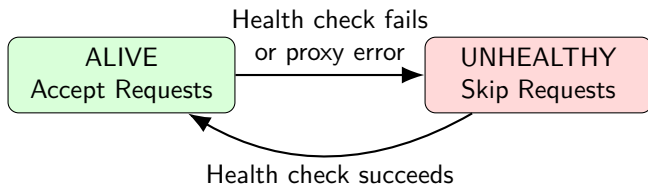
```
go lb.healthLoop(*healthInterval)
```

Conceptually:

```
for {  
    for _, backend := range backends {  
        GET backend.URL + "/health"  
  
        if request succeeds {  
            backend.Alive = true  
        } else {  
            backend.Alive = false  
        }  
    }  
  
    sleep(interval)  
}
```

The default health-check interval is approximately one second.

Health Checking State Transition



A failed backend is removed from normal scheduling until it becomes healthy again.

Step 6: Reverse Proxying in Go

Instead of manually creating a new HTTP request and copying the response, Go provides:

```
httputil.NewSingleHostReverseProxy(target)
```

The main idea:

```
target := backend.URL

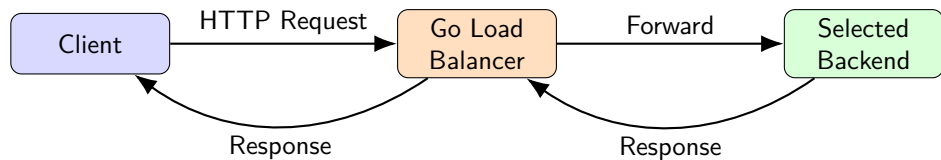
proxy :=
    httputil.NewSingleHostReverseProxy(target)

proxy.ServeHTTP(w, r)
```

The reverse proxy:

- 1 Receives the original request.
- 2 Changes the destination to the selected backend.
- 3 Sends the request.
- 4 Reads the backend response.
- 5 Returns the response to the original client.

Request Flow Through the Reverse Proxy



The client does not need to know which backend was selected.

Backend Error Handling

The reverse proxy can fail because:

- Backend is stopped.
- Network connection fails.
- Backend exceeds timeout.
- Backend returns a serious error.

The proxy error handler:

```
proxy.ErrorHandler =  
func(rw http.ResponseWriter,  
    req *http.Request,  
    err error) {  
  
    b.Alive.Store(false)  
    lb.metrics.BackendErrors.Add(1)  
    lb.metrics.Failed.Add(1)  
  
    http.Error(  
        rw,  
        "backend unavailable",  
        http.StatusBadGateway)  
}
```

The failed backend is immediately marked unhealthy.

Step 7: Configure Timeouts

A Load Balancer should not wait forever.

The code configures:

```
DialContext Timeout  
ResponseHeaderTimeout  
HTTP Client Timeout
```

Example:

```
backendTimeout := flag.Duration(  
    "backend-timeout",  
    800*time.Millisecond,  
    "backend request timeout")
```

If a backend delays for:

1 second

but the Load Balancer timeout is:

800 ms

the request can fail.

Step 8: Add Load Balancer Monitoring Endpoints

The Load Balancer exposes:

Endpoint	Purpose
/lb/health	Check if LB is alive
/lb/status	Backend health and state
/lb/metrics	Load Balancer metrics
/	Forward normal requests

Example:

```
GET http://SYS1:8080/lb/status
```

Possible response:

```
{
  "backends": [
    {"url": "SYS2", "alive": true},
    {"url": "SYS3", "alive": true},
    {"url": "SYS4", "alive": true}
  ]
}
```

Load Balancer Metrics

The Load Balancer stores counters such as:

```
type Metrics struct {  
    Total          atomic.Uint64  
    Success        atomic.Uint64  
    Failed         atomic.Uint64  
    BackendErrors  atomic.Uint64  
  
    LatencyMu sync.Mutex  
    Latencies []time.Duration  
}
```

Metrics answer questions such as:

- How many requests reached the LB?
- How many succeeded?
- How many failed?
- How many backend errors occurred?
- How long did requests take?

Step 9: Build a Concurrent Load Generator

The client must create artificial load independently from the four servers.

The client accepts:

```
-url  
-requests  
-concurrency  
-timeout  
-experiment  
-out  
-csv
```

Example:

```
.\client.exe '  
-url http://SYS1:8080 '  
-requests 5000 '  
-concurrency 40 '  
-experiment baseline
```

How the Worker Pool Works

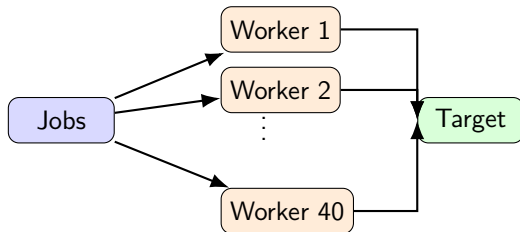
For:

Requests = 5000

and:

Concurrency = 40

we create approximately 40 concurrent workers:



Metric 1: Throughput

Throughput measures how many successful requests are completed per second.

$$\text{Throughput} = \frac{\text{Successful Requests}}{\text{Experiment Elapsed Time}}$$

Example:

$$\frac{4800 \text{ successful requests}}{10 \text{ seconds}} = 480 \text{ RPS}$$

The code calculates:

```
ThroughputRPS =  
    float64(successful) /  
    elapsed.Seconds()
```

Metric 2: Dropout Percentage

Dropout means the percentage of attempted requests that failed.

$$\text{Dropout} = \frac{\text{Failed Requests}}{\text{Total Requests}} \times 100$$

Example:

$$\frac{100}{5000} \times 100 = 2\%$$

A high dropout percentage may indicate:

- Backend failure
- Timeout
- Network problem
- Server overload

Metric 3: Latency Percentiles

Latency values are sorted:

```
sort.Slice(  
    latencies,  
    func(i, j int) bool {  
        return latencies[i] < latencies[j]  
    })
```

Then percentiles are calculated.

- **p50**: median or typical latency
- **p95**: 95% of requests are faster
- **p99**: shows tail latency

Example:

$$p50 = 20ms$$

$$p95 = 80ms$$

$$p99 = 400ms$$

The p99 value can reveal occasional slow requests.

Experiment Result Structure

Each experiment produces a result similar to:

```
{
  "experiment": "baseline",
  "requests": 5000,
  "concurrency": 40,
  "successful": 4980,
  "failed": 20,
  "throughput_rps": 420.5,
  "dropout_percent": 0.4,
  "p50_ms": 21.2,
  "p95_ms": 84.1,
  "p99_ms": 130.7
}
```

Results are saved in:

- JSON: one file per experiment
- CSV: cumulative comparison table

Experiment 1: Single Backend

For the first comparison:

Load Generator → Sys2 only

The Load Balancer can be configured with only one backend:

```
-backends http://SYS2:8081
```

Or the client can directly target Backend-1:

```
http://SYS2:8081/
```

Measure:

- Throughput
- Dropout
- p50
- p95
- p99

Experiment 2: Three Backends

Now configure the Load Balancer with:

```
-backends http://SYS2:8081,  
          http://SYS3:8081,  
          http://SYS4:8081
```

The topology becomes:

Load Generator $\rightarrow$ Load Balancer $\rightarrow \{B_1, B_2, B_3\}$

Run the same number of requests and the same concurrency.
Then compare the results fairly.

Expected Comparison Table

Students should produce a table similar to:

Experiment	Success	Failed	RPS	Dropout	p95	p99
1 Backend	4800	200	180	4.0%	110ms	190ms
3 Backends	4970	30	450	0.6%	70ms	120ms

Expected observation:

Under suitable load, multiple healthy backends should usually provide:

- Higher throughput
- Lower overload-related failures
- Better capacity

Actual results depend on the machines, network and workload.

Lab Exercise & Report Submission

Lab Exercise

- 1 Make a Load Balancer on your Sys1
- 2 Deploy backend on Sys2, Sys3 and Sys4
- 3 Make a Load Generator on your local system sending load to your load balancer
- 4 Measure metrics with Load Balancer using only Sys2
- 5 Measure metrics with Load Balancer using all three backends
- 6 Compare and add comparison table in your report
- 7 Use the **Messaging Project backend** as the backend for this implementation.
- 8 This Load Balancer implementation is an **individual project**.

Report Submission

- Student Name
- Roll Number
- Assigned Systems (IP/Hostname + Port)
- Load Balancer Code
- Comparison Table
- Relevant Screenshots

Submission

Submit a single PDF report in Google Classroom before the deadline.